Para la elaboración de este apartado, comenzamos con un proceso de exploración. Aunque ya estábamos familiarizados con algunas distancias semánticas, como la coseno y la euclidiana, decidimos ampliar el análisis a otras métricas. Nuestro objetivo era evaluar varias distancias y, con base en los resultados obtenidos, identificar cuál ofrecía la mejor precisión.

Resumen de distancias semánticas analizadas

A continuación, describimos las principales distancias estudiadas, sus rangos de valores y características:

1. Similitud coseno
    - Rango: [0, 1] (máxima similitud).
    - Descripción: Evalúa el ángulo entre dos vectores en un espacio n-dimensional, siendo ideal para comparar textos independientemente de su magnitud.

2. Distancia euclidiana
    - Rango: [0, ∞). Es 0 si los puntos son idénticos y crece a medida que se separan.
    - Descripción: Mide la distancia lineal entre dos puntos en un espacio euclidiano, siendo adecuada para diferencias absolutas en representaciones vectoriales.

3. Distancia de Hamming
    - Rango: [0, n], donde n es la longitud de las cadenas comparadas.
    - Descripción: Cuenta las posiciones en las que los caracteres de dos cadenas de igual longitud son diferentes. Es poco útil en términos semánticos.

4. Distancia de Levenshtein
    - Rango: [0, n+m], donde n y m son las longitudes de las cadenas.
    - Descripción: Calcula el número mínimo de operaciones (inserciones, eliminaciones o sustituciones) necesarias para transformar una cadena en otra, siendo más flexible que Hamming.

Dado que son varias las implementaciones, mencionaremos todas, pero profundizaremos en la que finalmente hemos elegido.

Implementaciones y resultados

1. *TF-IDF* con similitud coseno

   En primer lugar, probamos una técnica combinando el modelo *TF-IDF* con la similitud coseno. Esta técnica ofreció buenos resultados y es resultó sencilla de implementar usando la librería *sklearn*. Es especialmente útil para análisis de textos por su capacidad de medir orientaciones semánticas.

2. Distancia euclidiana

   En comparación, la similitud coseno mostró ser más versátil en la orientación de textos, pues quizás esta distancia requiere que los datos se gestionen en un cierto formato de representación concreto para lograr esa mejor precisión.

3. Distancia de Hamming y Levenshtein

   Ambas distancias tienen un enfoque más estructural que semántico, por lo que para este ejercicio concreto no son las más óptimas, aún así, consideramos su estudio y análisis. La *Hamming*, al depender de cadenas de igual longitud, es limitada. La *Levenshtein* es más flexible pero menos precisa para análisis semánticos profundos.

4. *Word2Vec* con *Word Mover's Distance (WMD)*

   En esta ocasión, utilizamos un modelo preentrenado de *Word2Vec*, que ya habíamos empleado en otros apartados. Implementamos una versión simplificada de *WMD* basada en la distancia euclidiana, la cual mostró resultados muy sólidos al evaluar similitudes semánticas entre textos. Este enfoque sobresalió por su capacidad para capturar relaciones profundas entre palabras, por ejemplo, reconocer palabras sinónimas o de significado muy similar, que es una de las principales claves para conseguir los mejores resultados.

   Dado que es el que mejores resultados nos ha ofrecido, a continuación, ofrecemos una explicación más extensa del mismo:

- El código utiliza el modelo preentrenado *Word2Vec* de *Google News*, que cuenta con representaciones vectoriales de 300 dimensiones para millones de palabras en inglés. Este modelo es proporcionado a través de la librería *Gensim* y es fundamental para calcular las similitudes semánticas.

- Función *word_movers_distance*: La función recibe como entrada dos textos y el modelo de *Word2Vec*, y realiza los siguientes pasos:

    1. Tokenización: Divide los textos en palabras y las convierte en minúsculas.

    2. Filtrado de vocabulario: Elimina las palabras que no están presentes en el vocabulario del modelo.

    3. Representación vectorial: Transforma las palabras en sus correspondientes representaciones vectoriales.

    4. Cálculo de distancia: Utiliza la función *cdist* de la librería *Scipy* para calcular las distancias euclidianas entre los vectores de las palabras.

- Características::

    - Si alguno de los textos no tiene palabras válidas dentro del vocabulario, la función devuelve un valor infinito (float("inf")).

    - El resultado final corresponde a la distancia mínima encontrada en la matriz de distancias calculada.

En resumen, este código proporciona una solución eficiente para calcular la similitud semántica entre textos, utilizando una metodología directa y eficaz basada en *Word2Vec* y la distancia euclidiana.

5. Modelos basados en BERT

También experimentamos con modelos preentrenados de *BERT*, obteniendo resultados robustos. Sin embargo, seguimos explorando alternativas más ligeras y específicas para comparación semántica.

6. Combinaciones con *Word2Vec* y *Doc2Vec*

Finalmente, de manera similar a como hicimos con la primera implementación con *TF-IDF*, probamos calcular la distancia semántica combinando

tanto un modelo *Word2Vec*, como luego, *Doc2Vec*, con la similitud coseno, obteniendo también excelentes resultados.

Conclusión

De todas las opciones analizadas, los métodos que mejor desempeño ofrecieron fueron:

- *Word2Vec* combinado con la distancia euclidiana.
- *Word2Vec* combinado con la similitud coseno.

Ambos métodos destacaron por su capacidad de capturar relaciones semánticas complejas y ofrecer resultados consistentes. En el notebook correspondiente se incluyen todos los códigos, comentarios y análisis para documentar las alternativas exploradas y justificar la elección final.